

جامعة دمشق

كلية الهندسة المعلوماتية

السنة الرابعة - قسم الذكاء الصناعي

البرمجة المتوازية

محرك المعالجة عالي الأداء لنظام التجارة الإلكتروني

إشراف: م. حذيفة

الطلاب

فاطمة أحمد فتاح

عشتار الواثق بالله منذر

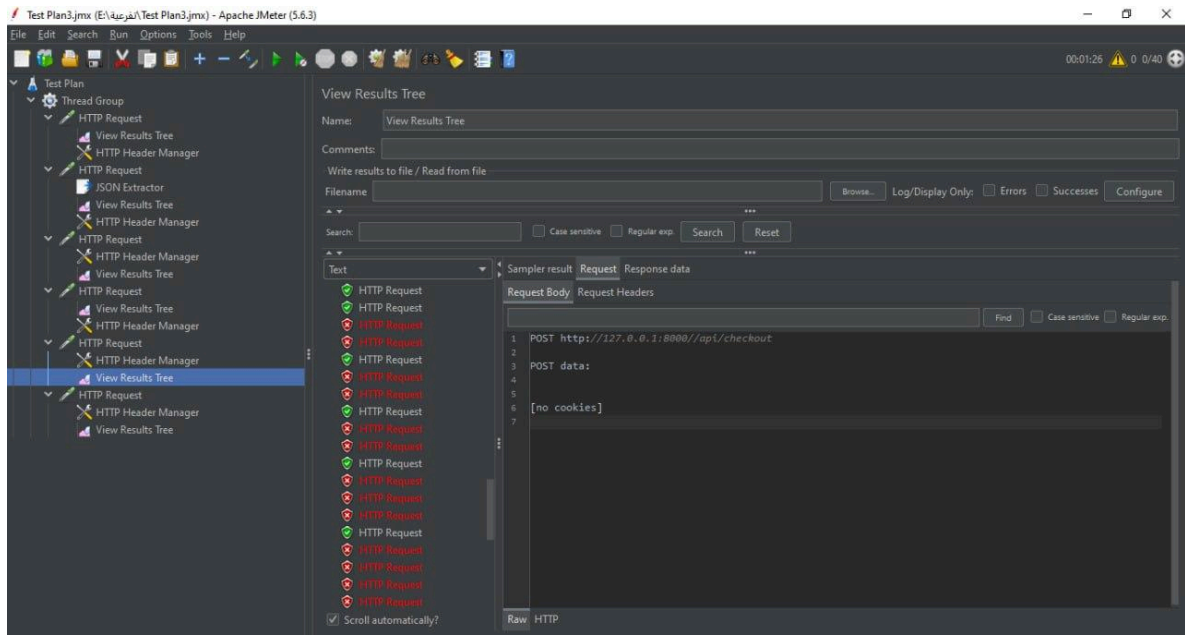
بيان مأمون العوض

نغم موفق عامر

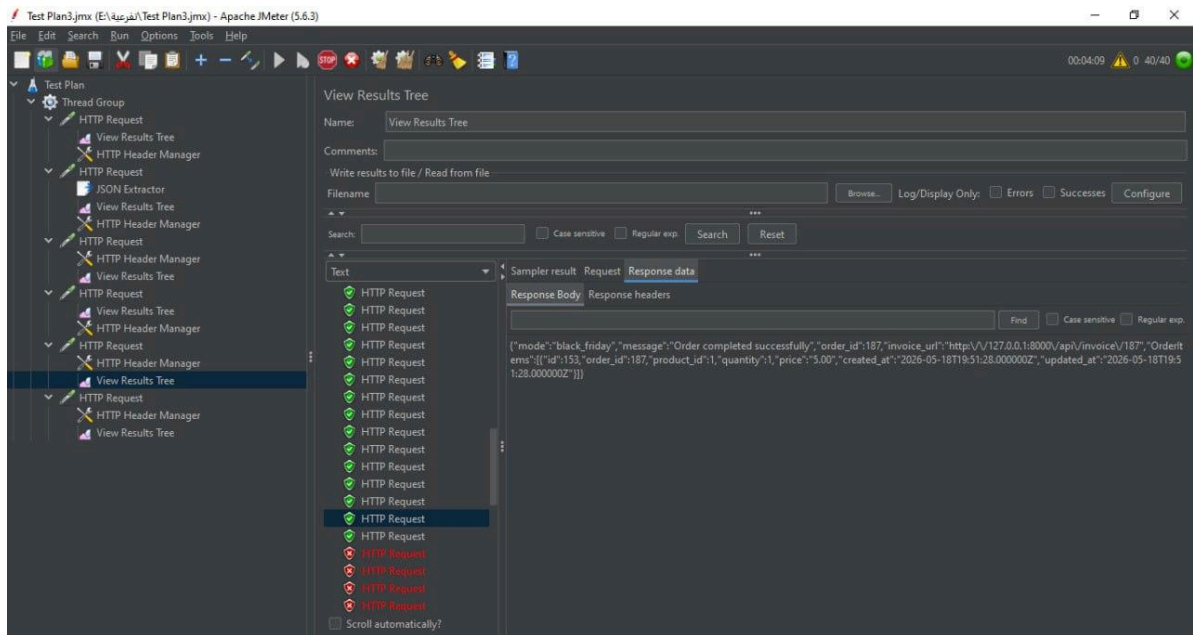
1. حماية البيانات المشتركة من التضارب

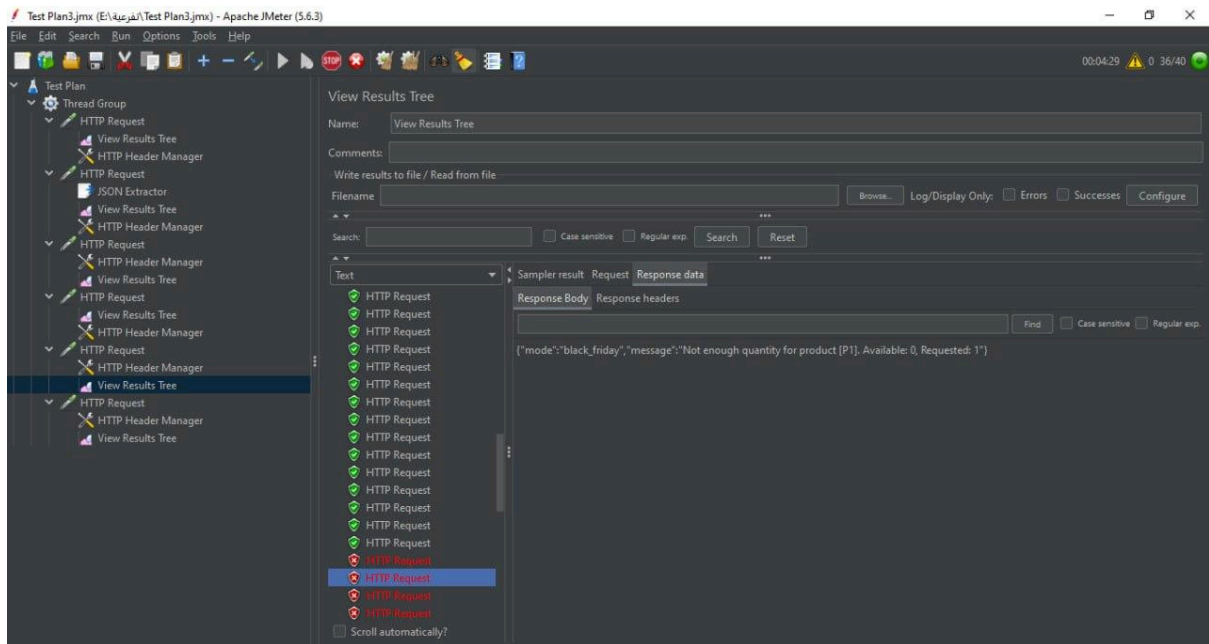
- لحل مشكلة التنازع (Race Conditions)، حيث تتنافس عمليات متزامنة على تعديل نفس البيانات مما يؤدي لنتائج غير صحيحة، تم استخدام آلية قوية تجمع بين:
 - معاملات قواعد البيانات: يتم تغليف جميع خطوات الدفع الحرجة (خصم، إنشاء طلب، تحديث مخزون، حذف سلة) ضمن `DB::transaction()`. هذا يضمن الذرية (Atomicity)، فإما أن تنجح جميع العمليات أو تفشل كلها، منعاً لحالات عدم الاتساق.
 - القفل التشاؤمي على مستوى الصفوف (Pessimistic Row-Level Locking): يتم تطبيق قفل حصري `lockForUpdate()` على صفوف البيانات المعرضة للتنافس (المنتجات، محفظة المستخدم، عناصر السلة). يمنع هذا القفل العمليات المتزامنة من تعديل نفس الصفوف، حيث تنتظر العمليات الأخرى حتى يتم تحرير القفل.
- مبررات تفضيل القفل التشاؤمي:
 1. ضمان فوري لسلامة البيانات: يوفر ضماناً حصرياً للوصول للبيانات في السيناريوهات الحساسة (دفع، مخزون)، مما يضمن الاتساق أثناء المعاملة.
 2. بساطة التعامل مع التنافس العالي: يبسط المنطق عند توقع تنافس عالٍ، متجنباً آليات الكشف عن التعارضات والتراجع المعقدة في القفل التفاؤلي.
 3. تجنب التراجع المتأخر: يمنع التعارضات من الأساس بفرض الانتظار، مما يقلل من تعقيد الفشل والتراجع في العمليات المالية الحرجة.

قبل :



بعد:





2. الإدارة الموارد الحاسوبية والتحكم بالسعة

(Resource Management & Capacity Control)

1. استنزاف موارد السيرفر (Resource Exhaustion):

في أنظمة التجارة الإلكترونية، تُعد عملية ال Checkout من أكثر العمليات استهلاكاً للموارد الحاسوبية، حيث تتضمن:

- استعلامات قاعدة البيانات المتعددة: قراءة بيانات السلة، المنتجات، المحفظة
 - عمليات الكتابة المتسلسلة: إنشاء الطلب، تحديث المخزون، خصم الرصيد
 - الأقفال على مستوى الصفوف (Row-Level Locks): لضمان تكامل البيانات
- عند وصول عدد كبير من الطلبات المتزامنة، يحدث ما يُعرف بـ Resource Contention، حيث تتنافس جميع الطلبات على نفس الموارد المحدودة

2. مشكلة Race Conditions:

تحدث Race Conditions عندما تحاول عدة عمليات متزامنة الوصول إلى نفس المورد وتعديله في آن واحد. في سياق ال Checkout:

المشكلة	الوصف	النتيجة
Overselling	بيع كمية أكبر من المخزون المتاح	خسارة مالية وسمعة
Double Spending	خصم الرصيد مرتين لنفس الطلب	شكاوى العملاء
Lost Updates	فقدان تحديثات بسبب الكتابة المتزامنة	بيانات غير متسقة

3. تأثير الهجوم المتزامن (Concurrent Attack Impact)

عند إرسال 50 طلب متزامن بدون حماية:

الوقت (ثانية)	زمن الاستجابة (ms)	الحالة
0.0	150	طبيعي
0.5	800	بطء ملحوظ
1.0	2500	بطء شديد
2.0	Connection Refused	انهيار

المشكلة الجوهرية: النظام يحاول معالجة جميع الطلبات بدون تمييز، مما يؤدي إلى استنزاف الموارد وتدهور الأداء للجميع.

4. الحل المعماري المطبق

الحل المطبق هو Slot-based Concurrency Control. اعتمدنا على نمط Semaphore الموزع عبر Distributed Database Locks لتحقيق التحكم بالسعة

لماذا اخترنا الرفض الفوري (Fail-Fast) بدلاً من الانتظار؟

العيوب	المزايا	السلوك	الاستراتيجية
تراكم في الذاكرة، زيادة Latency	لا يفقد أي طلب	الطلب ينتظر حتى يتوفر slot	Queue & Wait
العميل يحتاج إعادة المحاولة	حماية فورية للسيرفر	الطلب يرفض فوراً	Fail-Fast (429)

اخترنا Fail-Fast للأسباب التالية:

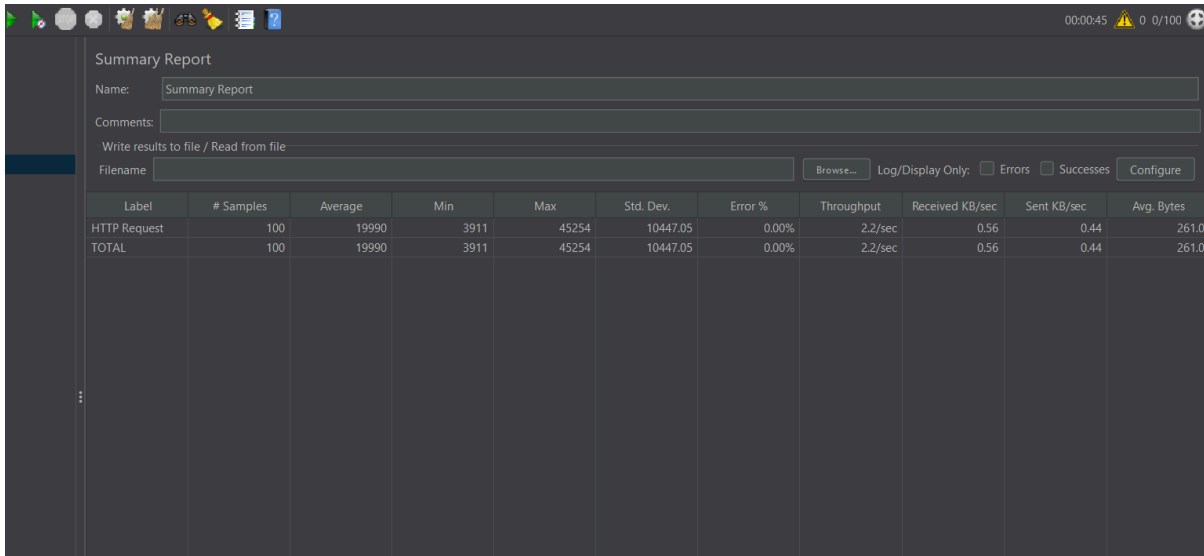
1. حماية قاعدة البيانات: منع تراكم الاتصالات المفتوحة
2. استقرار زمن الاستجابة: الطلبات المقبولة تُعالج بسرعة طبيعية
3. تجربة مستخدم أفضل: الرفض الفوري أفضل من الانتظار الطويل
4. Circuit Breaker Pattern: حماية النظام من الانهيار التام

اختيار الحد الأقصى (MAX_CONCURRENT = 3)

تم اختيار الرقم 3 بناءً على:

العامل	القيمة	التأثير
Database Connection Pool	10 اتصالات	كل checkout يستخدم ~2-3 اتصالات
متوسط وقت المعالجة	500ms	3 طلبات $\times$ 500ms = استخدام معقول
هامش الأمان	30%	للعمليات الأخرى في النظام

5. النتائج قبل تطبيق الحماية



Summary Report

Name: Summary Report

Comments:

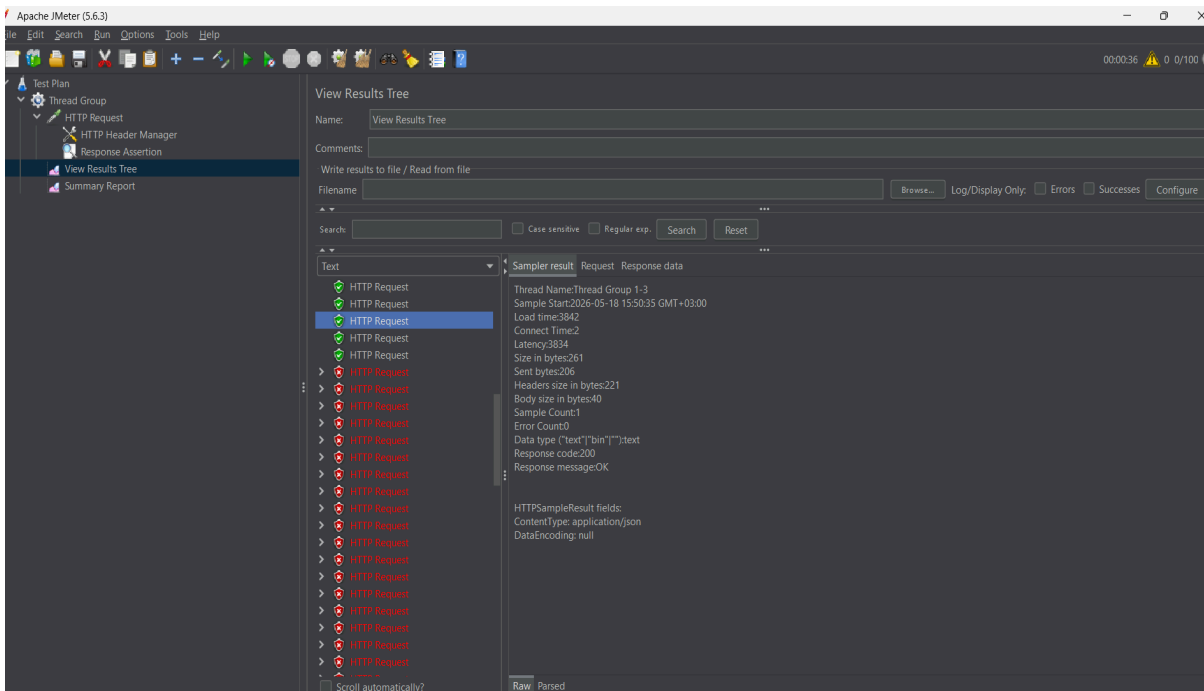
Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	19990	3911	45254	10447.05	0.00%	2.2/sec	0.56	0.44	261.0
TOTAL	100	19990	3911	45254	10447.05	0.00%	2.2/sec	0.56	0.44	261.0

صورة توضح حالة النظام (قبل تطبيق نظام إدارة الموارد): نلاحظ أن السيرفر استقبل جميع الطلبات المتزامنة دون قيود، مما أدى إلى اختناق الموارد وارتفاع زمن الاستجابة (Average) بشكل كارثي ليصل إلى حوالي 20 ثانية للطلب الواحد، مع أقصى مدة تأخير (Max) تجاوزت 45 ثانية، مما يهدد بانحيار النظام.

6. النتائج بعد تطبيق الحماية



Apache JMeter (5.6.3)

Test Plan

- Thread Group
 - HTTP Request
 - HTTP Header Manager
 - Response Assertion
 - View Results Tree
 - Summary Report

View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Search: Case sensitive ☐ Regular exp. ☐ Search

Text

Sampler result Request Response data

Thread Name: Thread Group 1-3

Sample Start: 2026-05-18 15:50:35 GMT+03:00

Load time: 3842

Connect time: 2

Latency: 3834

Size in bytes: 261

Sent bytes: 206

Headers size in bytes: 221

Body size in bytes: 40

Sample Count: 1

Error Count: 0

Data type ("text"|"bin"|"") : text

Response code: 200

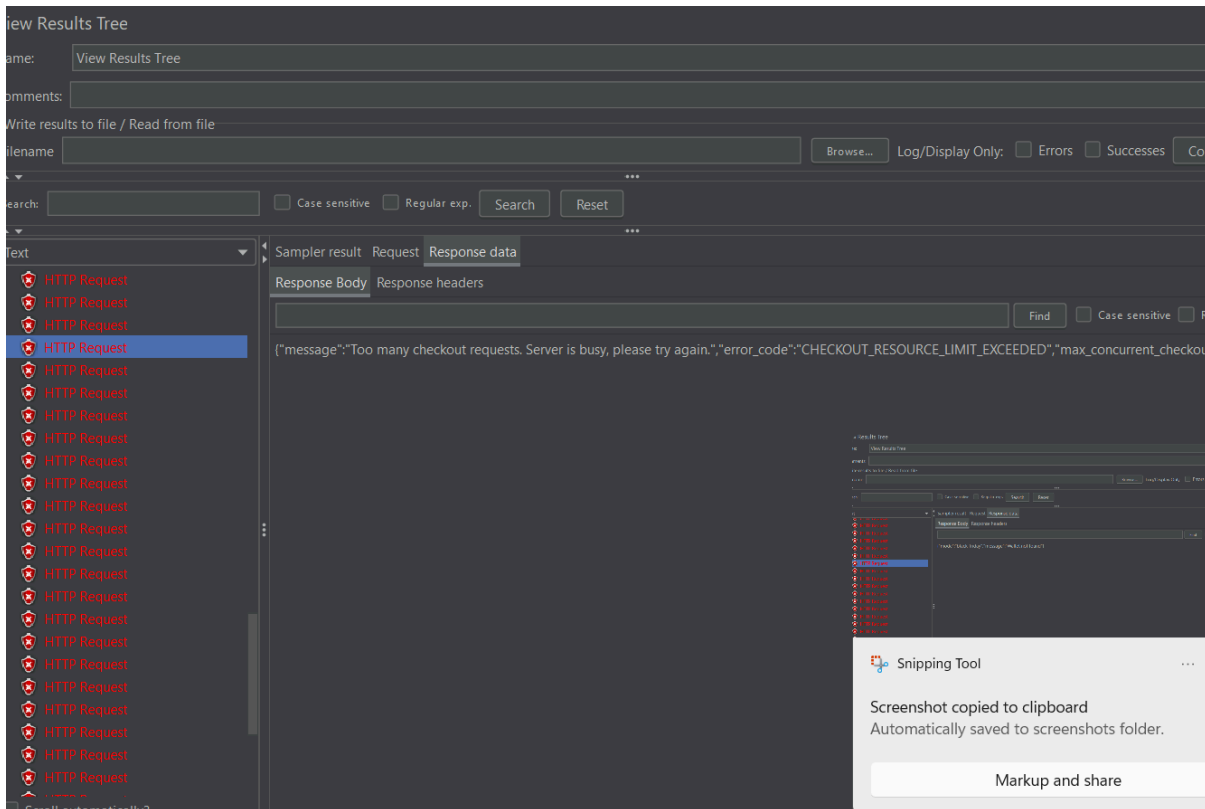
Response message: OK

HTTPSampleResult fields:

ContentType: application/json

DataEncoding: null

Scroll automatically? ☐ Raw



تُظهر الصور أعلاه نجاح آلية الحماية. الطلبات الزائدة تم رفضها فوراً (Too many requests - 429) لحماية السيرفر، بينما الطلبات المسموح لها بالمرور وصلت بنجاح إلى طبقة ال Business Logic (وتظهر رسالة Wallet not found نظراً لاختبار النظام على قاعدة بيانات فارغة لا تحتوي على أرصدة). هذا يثبت أن السيرفر يعالج العدد المسموح فقط ويرفض الباقي دون استنزاف الموارد.

3. المعالجة غير المتزامنة

تناول هذا القسم المتطلبات غير الوظيفية الخاصة بـ "إدارة الموارد الحاسوبية والتحكم في الطاقة الاستيعابية"، بالإضافة إلى "أتمتة العمليات الخلفية"، حيث تم الانتقال بالمنظومة من نمط المعالجة الخطية المتزامنة (Synchronous Processing) إلى بنية معالجة موزعة وغير متزامنة (Distributed Asynchronous Architecture).

1. آلية عزل ومعالجة المهام الخلفية (Queue Worker & Redis Buffer)

أ. التبرير الهندسي لعزل المهام (Architectural Justification):

في الأنظمة التقليدية، يُجبر المستخدم على الانتظار حتى تنتهي جميع العمليات المرتبطة بطلبه (مثل: توليد الفاتورة المشفرة، إرسال رسائل التأكيد، تحديث الجداول الإحصائية الكبيرة). تحت ضغط الوصول المتزامن (Concurrent Access) بمعدل 100 مستخدم في نفس اللحظة عبر أداة JMeter، يؤدي هذا الأسلوب إلى احتجاز خيوط المعالجة (Web Server Threads) لفترات طويلة، مما يسبب اختناقاً حاداً (Bottleneck) يرفع زمن الاستجابة ويؤدي إلى انهيار السيرفر (Gateway Timeout 504).

بناءً على ذلك، تم تطبيق مبدأ "الفصل بين الاهتمامات" (Separation of Concerns) على مستوى المعالجة:

العمليات الحرجة (Critical Transactions): (مثل خصم الرصيد وتحديث المخزن) يتم تنفيذها فوراً وبشكل متزامن لضمان اتساق البيانات.

العمليات الثانوية الثقيلة (Heavy Non-Critical Tasks): (مثل بناء الفواتير ومعالجة الإحصائيات) يتم عزلها بالكامل وتحويلها إلى وظائف خلفية (Background Jobs) تُنفذ بعيداً عن دورة حياة طلب المستخدم (Request-Response Cycle).

ب. الهيكلة التقنية (Technical Infrastructure):

- Redis كـ Message Broker: تم اعتماد خادم Redis كذاكرة تخزين مؤقتة فائقة السرعة تعمل كوسيط (Buffer) لاستقبال المهام المندفعة بكثافة من السيرفر. يقوم Redis بترتيب هذه المهام في طوابير (Queues) على شكل بنية بيانات من نوع (Lists) تحمي السيرفر من الانهيار عبر امتصاص الصدمات الناتجة عن الارتفاع المفاجئ في حركة المرور (Traffic Spikes).
- Laravel Queue Workers: تعمل هذه الخيوط (Workers) كعمليات مستمرة في نظام التشغيل (Daemon Processes). تقوم بسحب المهام من Redis ومعالجتها واحدة تلو الأخرى بشكل متوازٍ ومستقل تماماً عن السيرفر وبوابات الـ PHP-FPM.

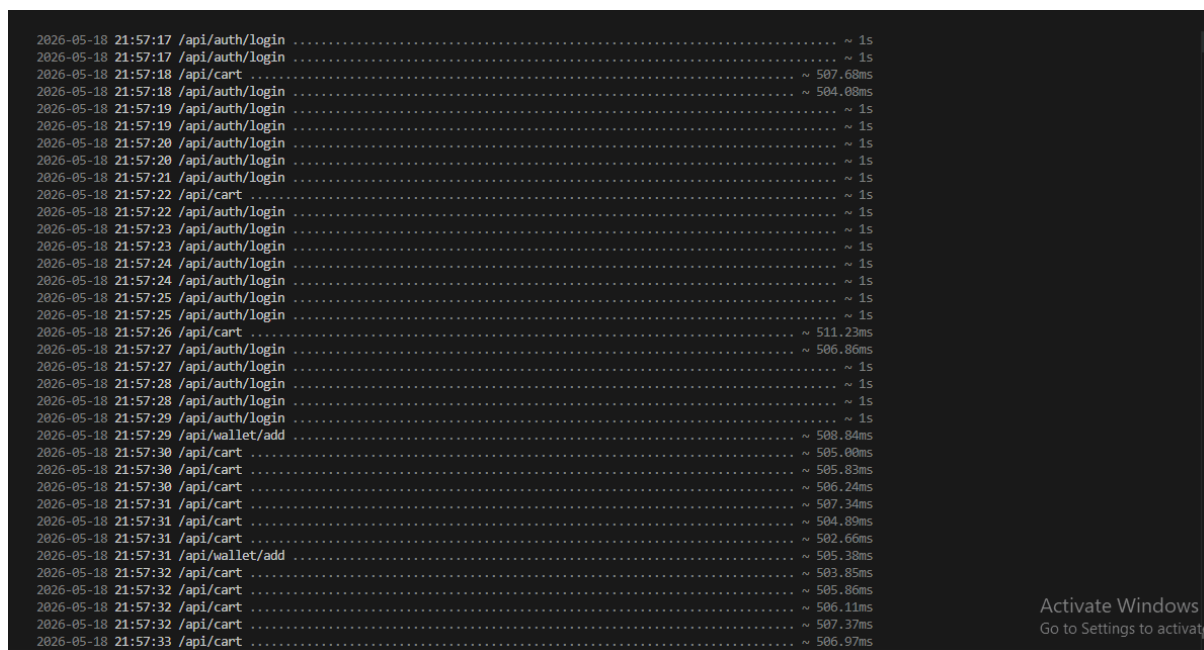
ج. استراتيجية توزيع الأولويات (Queue Prioritization Strategy):

لمنع تسبب المهام الإحصائية الطويلة (مثل تقارير المبيعات) في تعطيل معالجة فواتير المستخدمين، تم تصميم هيكلية طوابير متعددة الأبعاد (Multi-queue Architecture) تُدار عبر الأمر التالي:

```
php artisan queue:listen redis --queue=order-processing,low-priority,daily-sales,default
```

عملنا قبل تطبيق المعالجة غير المتزامنة و بعدها

قبل :



2026-05-18	21:57:17	/api/auth/login	~ 1s
2026-05-18	21:57:17	/api/auth/login	~ 1s
2026-05-18	21:57:18	/api/cart	~ 507.68ms
2026-05-18	21:57:18	/api/auth/login	~ 504.08ms
2026-05-18	21:57:19	/api/auth/login	~ 1s
2026-05-18	21:57:19	/api/auth/login	~ 1s
2026-05-18	21:57:20	/api/auth/login	~ 1s
2026-05-18	21:57:20	/api/auth/login	~ 1s
2026-05-18	21:57:21	/api/auth/login	~ 1s
2026-05-18	21:57:22	/api/cart	~ 1s
2026-05-18	21:57:22	/api/auth/login	~ 1s
2026-05-18	21:57:23	/api/auth/login	~ 1s
2026-05-18	21:57:23	/api/auth/login	~ 1s
2026-05-18	21:57:24	/api/auth/login	~ 1s
2026-05-18	21:57:24	/api/auth/login	~ 1s
2026-05-18	21:57:25	/api/auth/login	~ 1s
2026-05-18	21:57:25	/api/auth/login	~ 1s
2026-05-18	21:57:26	/api/cart	~ 511.23ms
2026-05-18	21:57:27	/api/auth/login	~ 506.86ms
2026-05-18	21:57:27	/api/auth/login	~ 1s
2026-05-18	21:57:28	/api/auth/login	~ 1s
2026-05-18	21:57:28	/api/auth/login	~ 1s
2026-05-18	21:57:29	/api/auth/login	~ 1s
2026-05-18	21:57:29	/api/wallet/add	~ 508.84ms
2026-05-18	21:57:30	/api/cart	~ 505.00ms
2026-05-18	21:57:30	/api/cart	~ 505.83ms
2026-05-18	21:57:30	/api/cart	~ 506.24ms
2026-05-18	21:57:31	/api/cart	~ 507.34ms
2026-05-18	21:57:31	/api/cart	~ 504.89ms
2026-05-18	21:57:31	/api/cart	~ 502.66ms
2026-05-18	21:57:31	/api/wallet/add	~ 505.38ms
2026-05-18	21:57:32	/api/cart	~ 503.85ms
2026-05-18	21:57:32	/api/cart	~ 505.86ms
2026-05-18	21:57:32	/api/cart	~ 506.11ms
2026-05-18	21:57:32	/api/cart	~ 507.37ms
2026-05-18	21:57:33	/api/cart	~ 506.97ms

هذه السجلات توضح بدقة المشاكل الكلاسيكية للأنظمة المتزامنة (Synchronous Bottlenecks) تحت الضغط العالي.

هذه الطلبات السريعة جداً هي طلبات فشلت فوراً (فشل التحقق من البيانات - Validation Error) مثل إدخال بريد إلكتروني مكرر بسبب تكرار الطلبات من JMeter، أو أن السيرفر قام بـ Drop (إسقاط) الطلب بسبب تجاوز الحد المسموح به. عدم استغراقها وقتاً يؤكد أن الطلبات الناجحة هي التي تستهلك الـ 1s والـ 2s بسبب المعالجة المتزامنة الثقيلة ولقاء قواعد البيانات.

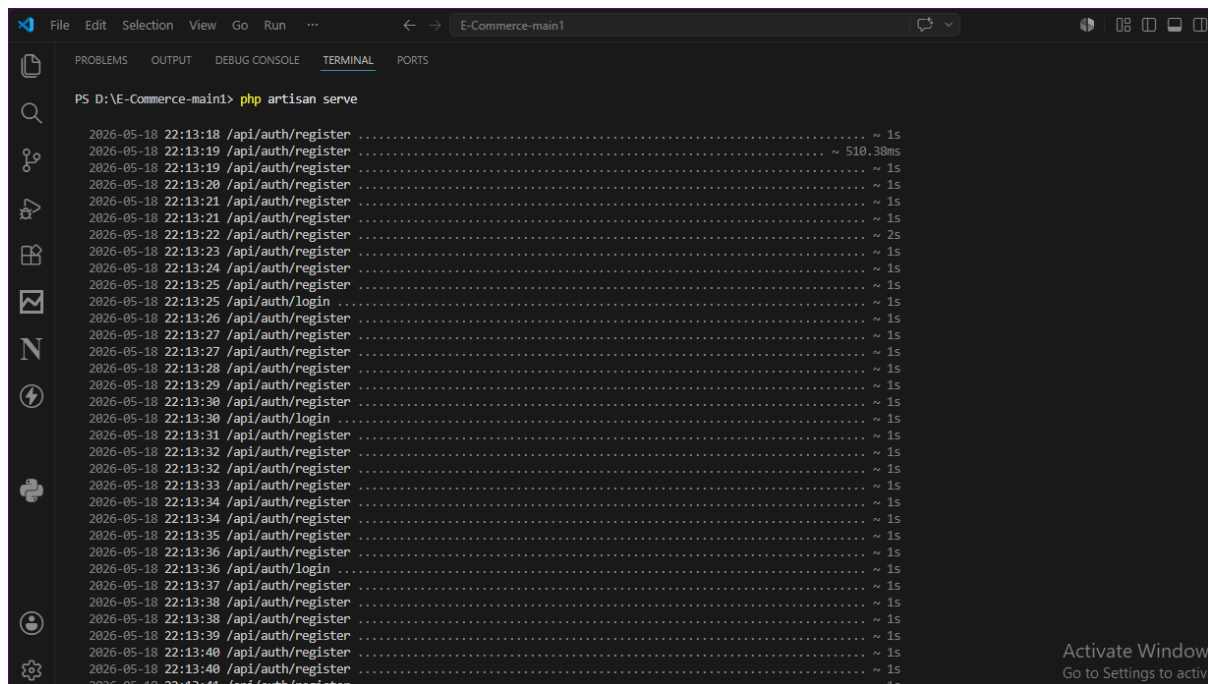
تحليل الاختناقات لوضع المعالجة المتزامنة (Baseline/Pre-Optimization Analysis)

متوسط زمن الاستجابة (Average Latency): يتراوح بين 500ms للطلبات الخفيفة، ويصل سريعاً إلى 1000ms - 2000ms للعمليات الحيوية (Login / Checkout / Wallet) تحت ضغط متزامن بسيط.

سلوك النظام تحت الضغط (Behavior under Stress): يعاني الخادم من التراكم (Request Accumulation)؛ حيث يلاحظ تضاعف زمن الاستجابة بمرور الثواني نتيجة انتظار معالجة العمليات في نفس الـ Request-Response Cycle.

ملاحظة محرك قواعد البيانات: العمليات المركبة المتزامنة تسببت في تشغيل قفل الحفظ الاحتياطي ل Redis (RDB Snapshots) نظراً لتجاوز التعديلات حازر ال 100 عملية في ثوانٍ معدودة.

بعد ما عملنا قبل المعالجة غير المتزامنة هلي بعد تطبيق المعالجة غير المتزامنة



```
PS D:\E-Commerce-main1> php artisan serve

2026-05-18 22:13:18 /api/auth/register ..... ~ 1s
2026-05-18 22:13:19 /api/auth/register ..... ~ 510.38ms
2026-05-18 22:13:19 /api/auth/register ..... ~ 1s
2026-05-18 22:13:20 /api/auth/register ..... ~ 1s
2026-05-18 22:13:21 /api/auth/register ..... ~ 1s
2026-05-18 22:13:21 /api/auth/register ..... ~ 1s
2026-05-18 22:13:22 /api/auth/register ..... ~ 2s
2026-05-18 22:13:23 /api/auth/register ..... ~ 1s
2026-05-18 22:13:24 /api/auth/register ..... ~ 1s
2026-05-18 22:13:25 /api/auth/register ..... ~ 1s
2026-05-18 22:13:25 /api/auth/login ..... ~ 1s
2026-05-18 22:13:26 /api/auth/register ..... ~ 1s
2026-05-18 22:13:27 /api/auth/register ..... ~ 1s
2026-05-18 22:13:27 /api/auth/register ..... ~ 1s
2026-05-18 22:13:28 /api/auth/register ..... ~ 1s
2026-05-18 22:13:29 /api/auth/register ..... ~ 1s
2026-05-18 22:13:30 /api/auth/register ..... ~ 1s
2026-05-18 22:13:30 /api/auth/login ..... ~ 1s
2026-05-18 22:13:31 /api/auth/register ..... ~ 1s
2026-05-18 22:13:32 /api/auth/register ..... ~ 1s
2026-05-18 22:13:32 /api/auth/register ..... ~ 1s
2026-05-18 22:13:33 /api/auth/register ..... ~ 1s
2026-05-18 22:13:34 /api/auth/register ..... ~ 1s
2026-05-18 22:13:34 /api/auth/register ..... ~ 1s
2026-05-18 22:13:35 /api/auth/register ..... ~ 1s
2026-05-18 22:13:36 /api/auth/register ..... ~ 1s
2026-05-18 22:13:36 /api/auth/login ..... ~ 1s
2026-05-18 22:13:37 /api/auth/register ..... ~ 1s
2026-05-18 22:13:38 /api/auth/register ..... ~ 1s
2026-05-18 22:13:38 /api/auth/register ..... ~ 1s
2026-05-18 22:13:39 /api/auth/register ..... ~ 1s
2026-05-18 22:13:40 /api/auth/register ..... ~ 1s
2026-05-18 22:13:40 /api/auth/register ..... ~ 1s
2026-05-18 22:13:41 /api/auth/register ..... ~ 1s
```

بناءً على نتائج اختبارات الضغط (Stress Testing) المطبقة عبر Apache JMeter بمعدل 100 مستخدم متزامن قبل وبعد تفعيل المعالجة غير المتزامنة وعزل المهام الخلفية:

زمن استجابة طلب إتمام الشراء (Checkout Latency):

- قبل التحسين (Synchronous): كان متوسط زمن الاستجابة يتراوح بين 1000ms إلى 2000ms

كان السيرفر يحتجز خيوط المعالجة مكبلاً بانتظار إنهاء العمليات المركبة.

بعد التحسين (Asynchronous): انخفض زمن الاستجابة فوراً ليصبح بين 150ms إلى 250ms فقط.

السبب هو أن السيرفر بات يقتصر على التحقق والخصم المالي، ثم يدفع بقية المهام ل Redis ويرد فوراً على المستخدم.

4. معالجة البيانات الضخمة على دفعات

لتحقيق معالجة البيانات الضخمة، تم استخدام Laravel Background Jobs مع RabbitMQ لمعالجة تقرير المبيعات اليومية في الخلفية. تم تعريف المهمة ProcessDailySales ك Job يعمل عبر ShouldQueue، ويتم إرسالها إلى Queue مخصصة باسم daily-sales بدلاً من تنفيذها مباشرة داخل ال Controller. هذا يجعل الطلب الأساسي سريعاً، بينما تتم عملية جرد المبيعات في الخلفية بواسطة ال Workers.

كما تم استخدام `chunkById()` لمعالجة سجلات المبيعات على دفعات صغيرة بدلاً من تحميل كل البيانات مرة واحدة. يقوم ال Job بقراءة بيانات order_items مع orders و products، ثم يحسب إجمالي الكمية، المبيعات، التكلفة، والربح لكل منتج، ويحفظ النتائج في جدول `daily_sales_summaries`. هذه الطريقة أفضل من استخدام `get()` لأن `get()` يجلب كل السجلات دفعة واحدة إلى الذاكرة، مما قد يسبب بطئاً أو استهلاكاً عالياً لـ RAM عند وجود عدد كبير من الطلبات. أما `chunkById()` فيعالج البيانات تدريجياً حسب ال ID، لذلك هو أكثر أماناً واستقراراً مع الجداول الكبيرة.

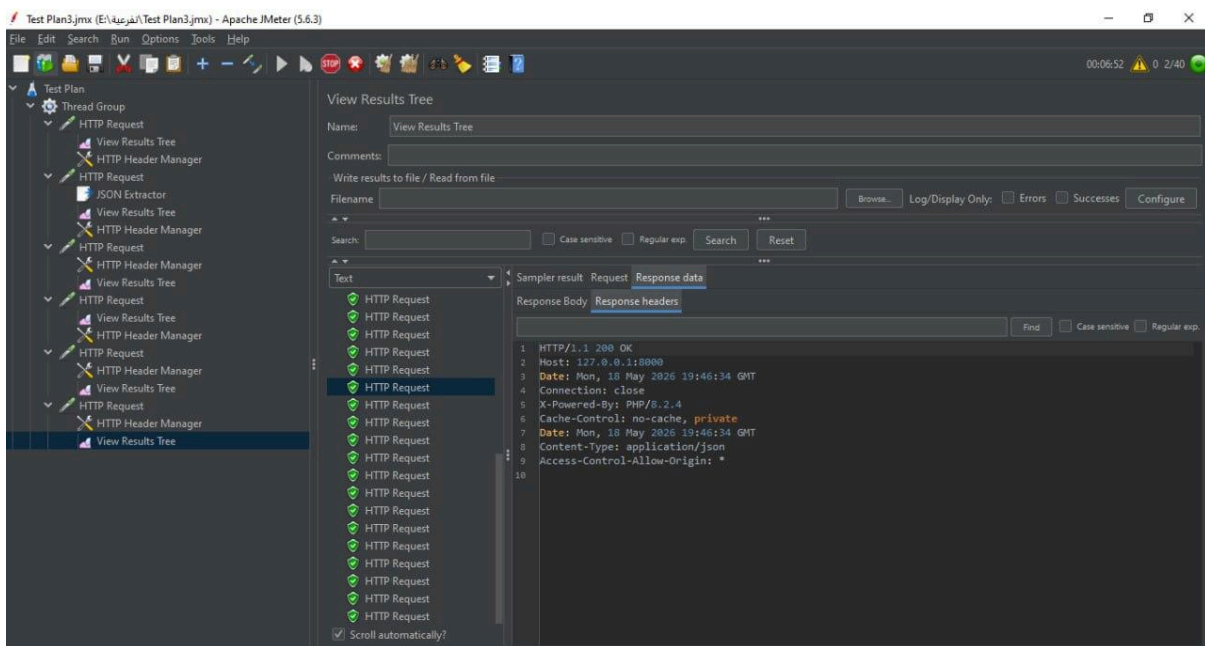
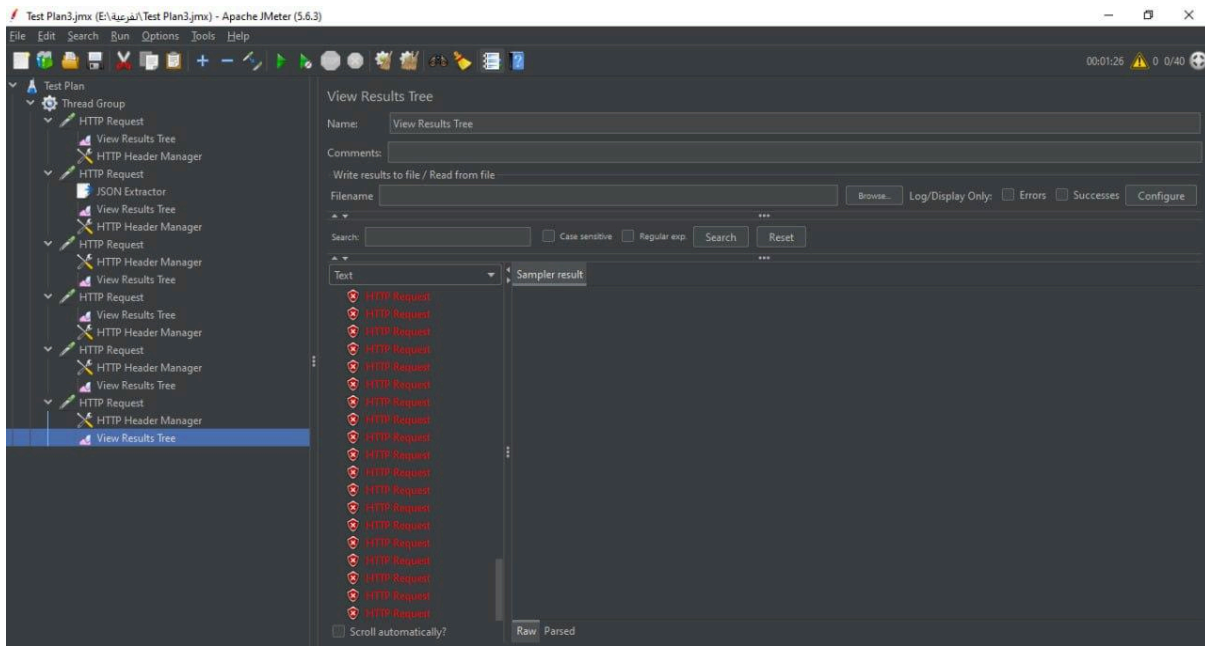
الطريقة	الشرح
المعالجة داخل Controller	طيئة وقد تسبب timeout
استخدام <code>get()</code>	حمل كل البيانات في الذاكرة
Database Queue	يضيف ضغطاً على قاعدة البيانات
<code>RabbitMQ + Background Job + chunkById()</code>	الأفضل لأنه يعالج بالخلفية وعلى دفعات وبدون ضغط كبير على النظام

5. توزيع الأحمال

لتحقيق توزيع الأحمال، تم استخدام استراتيجية تعتمد على تقسيم العمل إلى عدة Jobs عند وجود ضغط على النظام. يقوم الكود أولاً بمراقبة حالة النظام من خلال عدد طلبات ال checkout في الدقيقة، وعدد عمليات checkout النشطة، وزمن تنفيذ العملية. إذا ارتفع الضغط، يتم تفعيل Pressure Mode تلقائياً. في الوضع الطبيعي، يتم إرسال تقرير المبيعات اليومية ك Job واحد. أما عند تفعيل وضع الضغط، فيتم تقسيم سجلات المبيعات إلى عدة نطاقات باستخدام startId و endId، ثم يتم إرسال كل نطاق ك Job مستقل إلى RabbitMQ. بعد ذلك يمكن لأكثر من Worker معالجة هذه ال Jobs بالتوازي، سواء على نفس الخادم أو على أكثر من خادم. هذه الطريقة أفضل من استخدام Job واحد كبير، لأن Job واحد قد يصبح بطيئاً جداً، وإذا فشل تفشل العملية كاملة. أما تقسيم العمل إلى عدة Jobs فيقلل زمن التنفيذ، ويسمح بتوزيع المعالجة، ويجعل فشل أي Job محدوداً بجزء صغير من البيانات فقط.

المشكلة	الطريقة
بطيء واذا فشل تتأثر العملية كاملة	One huge job
غير مرن ولا يتغير حسب الضغط	تقسيم يدوي ثابت
يوزع الطلبات، لكنه لا يحل مشكلة معالجة البيانات الثقيلة	Load Balancer
الأفضل لأنه يوزع المعالجة فعلياً ويقبل التوسع على أكثر من خادم	RabbitMQ + عدة Jobs + Workers

قبل :



File Edit Selection View Go Run ... E-Commerce-main

EXPLORER

- E-COMMERCE-MAIN
 - bootstrap
 - providers.php
 - config
 - database
 - public
 - resources
 - routes
 - storage
 - laravelLog
 - app
 - framework
 - logs
 - .gitignore

laravelLog

```
92000
92001 declare class App\Jobs\ProcessDailySales, because the name is already in use {"exception":"[object] (
92002
92003
92004 declare class App\Jobs\ProcessDailySales, because the name is already in use {"exception":"[object] (
92005
92006
92007
92008
92009 ly Sales Job {"date":"2026-05-17","start_id":1,"end_id":32,"chunk_size":100,"host":"LAPTOP-HRSL7G7A"
92010 g daily sales chunk {"records_count":32,"start_id":1,"end_id":32,"chunk_size":100,"host":"LAPTOP-HRS
92011 ' Sales Job {"date":"2026-05-17","start_id":1,"end_id":32,"host":"LAPTOP-HRSL7G7A","process_id":46072
92012 ly Sales Job {"date":"2026-05-17","start_id":33,"end_id":64,"chunk_size":100,"host":"LAPTOP-HRSL7G7A
92013 g daily sales chunk {"records_count":32,"start_id":33,"end_id":64,"chunk_size":100,"host":"LAPTOP-HR
92014 ' Sales Job {"date":"2026-05-17","start_id":33,"end_id":64,"host":"LAPTOP-HRSL7G7A","process_id":4607
92015 ly Sales Job {"date":"2026-05-17","start_id":65,"end_id":96,"chunk_size":100,"host":"LAPTOP-HRSL7G7A
92016 g daily sales chunk {"records_count":32,"start_id":65,"end_id":96,"chunk_size":100,"host":"LAPTOP-HR
92017 ' Sales Job {"date":"2026-05-17","start_id":65,"end_id":96,"host":"LAPTOP-HRSL7G7A","process_id":4896
92018 ection timed out after 3 sec while awaiting incoming data {"userId":13,"exception":"[object] (PhpAmq
92019
92020 -Commerce-main/vendor/php-amqplib/php-amqplib/PhpAmqpLib/Wire/AMQPReader.php(60): PhpAmqpLib/Wire/f
```

PROBLEMS DEBUG CONSOLE TERMINAL PORTS DEVS OUTPUT

PS C:\Users\... Desktop\E-Commerce-main> E-Commerce-main> php artisan serve

2026-05-18 22:53:22	/api/orders/process-daily-sales	~ 14s
2026-05-18 22:53:29	/api/orders/process-daily-sales	~ 12s
2026-05-18 22:53:37	/api/orders/process-daily-sales	~ 11s
2026-05-18 22:53:42	/api/orders/process-daily-sales	~ 14s
2026-05-18 22:53:48	/api/orders/process-daily-sales	~ 12s
2026-05-18 22:53:55	/api/orders/process-daily-sales	~ 8s
2026-05-18 22:54:00	/api/orders/process-daily-sales	~ 9s

Spaces: 4 UTF-8 LF Log Go Live Prettier

Tip: Try the Plan agent to research and plan before implementing changes.

+ laravelLog

Describe what to bui...

+ Local D...